Deploying the Portfolio to Vercel

Sachal Chandio · React (Vite) SPA + Flask serverless API · free Hobby tier

Status: the Vercel config is built, verified, committed and **pushed to GitHub main** (commit `fea0c1b`). What remains is one login on your side to import & deploy. No credit card required.

1 · What was configured (already done)

Vercel serves the React build from its CDN and runs the Flask API as a Python serverless function. These files were added to the repo:

File	Purpose
`vercel.json`	Build the Vite app from frontend/, serve frontend/dist, route /api/* + /healthz to the function, SPA fallback to index.html.
`api/index.py`	Serverless entry — puts backend/ on sys.path and re-exports the existing Flask app (no change to backend/app.py).
`requirements.txt`	Flask + flask-cors (gunicorn dropped — Vercel invokes the WSGI app directly).
`.python-version`	Pins the function runtime to Python 3.12.
`frontend/public/Sachal_Chandio_Resume.pdf`	Resume served as a static CDN asset; the 4 in-app links now point here with download.

2 · Deploy — Dashboard import (recommended · one login)

1. Go to `vercel.com` and Sign up / Log in with GitHub (this single login covers both auth and repo access — no card for the Hobby tier).
2. Click Add New → Project (top-right).
3. Find Sachal-Resume in the repo list and click Import.
4. On the configure screen, leave Root Directory blank (the repo root) — do NOT choose `frontend`. Framework auto-detects as Vite; leave Build / Output / Install on default (they come from vercel.json).
5. Click Deploy. Vercel runs the Vite build, detects requirements.txt, and ships api/index.py as a Python function.
6. When it finishes, open the production URL (`*.vercel.app`) and run the smoke tests in section 4.
7. Done — every push to `main` now auto-deploys; branches/PRs get preview URLs.

Why Root Directory must stay blank: setting it to `frontend` hides the root `api/` folder, so the Flask function would never deploy.

3 · Deploy — CLI with a token (scripted alternative)

If you'd rather it run non-interactively (or have me run it), create a token at `vercel.com/account/tokens`, then from the repo root:

```
npm i -g vercel
$env:VERCEL_TOKEN = "vcp_xxxxxxxxxxxxxxxxxxxx"
vercel deploy --prod --yes --token $env:VERCEL_TOKEN
```

The first run links/creates the project and prints the live URL. Note: a CLI deploy does not install the push-to-deploy webhook — use the dashboard path (section 2) for auto-deploys.

4 · After deploy — verify (do not skip)

Vercel's runtime can't be tested locally, so confirm on the live URL:

```
/ -> homepage loads (hero, 3D)
/off-duty /projects -> client routes load; refresh on a deep link still works
/api/portfolio -> JSON responds (also /api/off-duty, /api/projects, /healthz)
Resume button -> downloads Sachal_Chandio_Resume.pdf
Contact form -> submit returns a success message (POST /api/contact)
```

5 · Notes & limitations

- Cold starts: after idle, the first /api/* hit pays a short Python cold start. The static SPA + PDF are CDN-served and unaffected.
- Hobby tier is free and non-commercial; limits (100 GB bandwidth, 1M function calls/mo) are far beyond a portfolio's needs.
- frontend/dist is gitignored on purpose — Vercel rebuilds from source. If the build (tsc + vite) ever fails, the whole deploy fails, so keep it green.
- The Dockerfile / k8s / render.yaml in the repo describe a different (single-image) deployment and are ignored by Vercel — no conflict.
- To personalize the hero photo, drop your headshot at frontend/public/profile.jpg and push; the next deploy picks it up.

Generated for Sachal Chandio · repo: github.com/sachalchandio/Sachal-Resume